ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 13

Organizar el código y controlar los errores

- include y require: montar la web por piezas
- Rutas que no se rompen: __DIR__
- Una web con cabecera y pie comunes
- Leer un mensaje de error sin miedo
- Depurar: buscar el fallo con método
- Excepciones: try, catch y finally
- Errores en vuestro ordenador y en internet



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Nadie escribe una web entera en un único archivo de diez mil líneas. Bueno... alguno lo ha intentado, y os aseguro que no acaba bien. 😅 Hoy aprendemos a **dividir el proyecto en piezas** (cabecera, pie, funciones) y a montarlas con `require`.

La segunda parte va de algo que os acompañará toda la vida: **los errores**. Vamos a aprender a leerlos sin miedo, a buscar fallos con método y a controlar los problemas con **excepciones**, para que vuestra web no le enseñe al visitante una pantalla llena de mensajes en rojo.

Con esta clase cerramos la parte de **PHP paso a paso**. Después vienen las bases de datos y la programación orientada a objetos.

1

📘 **include y require: montar la web por piezas**



2

⚠️ **Rutas que no se rompen: __DIR__**



3

👌 **Una web con cabecera y pie comunes**



4

🤔 **Leer un mensaje de error sin miedo**



5

🔍 **Depurar: buscar el fallo con método**



6

🚀 **Excepciones: try, catch y finally**



7

👍 **Errores en vuestro ordenador y en internet**



8

🚀 **Ejercicios**



9

👋 **Lo que tenéis que llevaros de hoy**



🧩 Organizar el código



CLASE 13 · PUNTO 1

📘 include y require: montar la web por piezas

Con `include` y `require`, PHP **copia y pega** el contenido de otro archivo justo en esa línea, como si lo hubierais escrito ahí [1]. Hay cuatro variantes, y solo se diferencian en dos cosas:

Sentencia	Si el archivo no existe…	Si ya se había cargado…	Cuándo usarla
`include`	Aviso (*Warning*) y **sigue**	Lo vuelve a cargar	Casi nunca
`require`	Error fatal y **se para**	Lo vuelve a cargar	Trozos de HTML: cabecera, pie…
`include_once`	Aviso y sigue	No hace nada	Casi nunca
`require_once`	Error fatal y se para	**No hace nada**	Archivos de **funciones** y, más adelante, de **clases**

¿Por qué `require_once` para las funciones? Porque si un archivo de funciones se carga dos veces, PHP intenta declarar dos veces la misma función y se para con «*Cannot redeclare function*». Con `_once`, la segunda vez simplemente no hace nada.

📄 Ejemplo: `01-include-require.php`



🎯 **Mi regla:** `require_once` para archivos de funciones o configuración; `require` para trozos de HTML que se repiten. `include`, prácticamente nunca: si un archivo es necesario, mejor que el programa se pare a que siga funcionando a medias.



CLASE 13 · PUNTO 2

⚠️ Rutas que no se rompen: __DIR__

Esto parece correcto:

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
require "includes/funciones.php";
```

Y a veces funciona... y a veces no, según desde qué carpeta se esté ejecutando el script. Es de esos errores que vuelven loco a cualquiera.

La solución es usar `__DIR__`, una constante especial que siempre contiene **la carpeta del archivo en el que está escrita** [2]:

ejemplo.php — php-paso-a-paso/clase13 — Visual Studio Code

ejemplo.php

```php
<?php
require_once __DIR__ . "/includes/funciones.php";
```

Así la ruta es siempre completa (por ejemplo, `C:\laragon\www\php-paso-a-paso\clase13\includes\funciones.php`) y funciona la mováis donde la mováis.



🎯 Acostumbraos desde hoy: **todas** vuestras inclusiones con `__DIR__ . "/..."`.



CLASE 13 · PUNTO 3

👌 Una web con cabecera y pie comunes

Imaginad una web con veinte páginas. Todas llevan el mismo menú. Un día hay que añadir un enlace al menú... ¿en veinte archivos? No: lo sacamos a un archivo aparte y cada página lo incluye.

Esquema

```
web/
├── includes/
│   ├── funciones.php   ← e(), euros()… todas las funciones
│   ├── cabecera.php    ← <html>, <head>, el menú y la apertura de <main>
│   └── pie.php         ← cierre de <main>, <footer> y </html>
├── index.php
├── productos.php
└── contacto.php
```

Cada página queda cortita y solo contiene lo que la hace distinta:

— php-paso-a-paso/clase13 — Visual Studio Code

```php
<?php
declare(strict_types=1);

require_once __DIR__ . "/includes/funciones.php";

$titulo = "Productos";      // la cabecera usará esta variable en el <title>
$pagina = "productos";      // y esta para marcar el enlace activo del menú

require __DIR__ . "/includes/cabecera.php";
?>

<h1>Productos</h1>
<p>Aquí, solo el contenido de esta página.</p>

<?php require __DIR__ . "/includes/pie.php"; ?>
```

Fijaos en un detalle importante: los archivos incluidos **ven las variables** de la página que los incluye. Por eso la cabecera puede usar `$titulo` y `$pagina`. Tiene sentido: `require` pega el código justo ahí, como si estuviera escrito en la propia página.

📄 Ejemplo: la carpeta `web/`. Abrid `http://php-paso-a-paso.test/clase13/web/` y navegad por el menú.



💡 Lo que acabáis de hacer es la idea básica de las **plantillas** (*layouts*). Laravel lo hace de una forma más elegante con Blade, pero la idea es exactamente esta.

🐞 Errores



CLASE 13 · PUNTO 4

🤔 Leer un mensaje de error sin miedo

Un mensaje de error **no es un insulto**: es una pista. Y casi siempre os dice exactamente qué pasa y dónde. Mirad uno:

Esquema

```
Warning: Undefined array key "email" in C:\laragon\www\php-paso-a-paso\clase13\perfil.php on
line 12
   |                |                                  |                          |
  tipo          qué ha pasado                    en qué archivo              en qué
línea
```

Leedlo siempre en ese orden: **tipo → qué → dónde**. Estos son los tipos que más veréis [3]:

Tipo	¿Se para PHP?	Ejemplo típico	Mensaje
Parse error	Ni siquiera empieza	Falta un `;` o una llave	`syntax error, unexpected...`
Fatal error	Sí, en esa línea	Llamar a una función que no existe	`Call to undefined function`
TypeError	Sí	Pasar un texto donde se espera un `int`	`must be of type int, string given`
Warning	No, sigue	Leer una clave que no existe	`Undefined array key`
Deprecated	No	Usar algo que desaparecerá en futuras versiones	`... is deprecated`

📄 Ejemplo: `02-tipos-de-errores.php` . Tiene cada error comentado: quitadle las `//` de uno en uno, leed el mensaje y volved a comentarlo.



😅 **Truco con los *parse error*:** PHP suele señalar la línea **siguiente** a la del fallo, porque es ahí donde se da cuenta de que algo no cuadra. Si la línea que os indica parece correcta, mirad la de arriba.



CLASE 13 · PUNTO 5

🔍 Depurar: buscar el fallo con método

Depurar es buscar por qué el programa no hace lo que esperáis. Y la regla número uno es: **no adivinéis, mirad**. En vez de cambiar cosas al azar, mirad qué valor tienen de verdad las variables.

Herramienta	Qué hace	Cuándo
`var_dump($x)`	Muestra el tipo y el valor [4]	Para ver **exactamente** qué hay
`print_r($x)`	Muestra arrays de forma legible	Para arrays grandes
`exit;`	Para el programa en ese punto	Para ver hasta dónde llega

En el ejemplo os he preparado una funcioncita muy práctica, `dd()` («*dump and die*»: muestra y para), que también existe en Laravel:



📄 Ejemplo: `03-depurar.php`

Mi método en cuatro pasos

1. **Reproducid el fallo.** ¿Con qué datos ocurre? ¿Siempre o solo a veces?
2. **Acotad.** Poned un `dd()` a mitad de camino. ¿El valor ya está mal ahí? Entonces el fallo está antes. ¿Está bien? Está después.
3. **Corregid** una sola cosa cada vez.
4. **Quitad** los `dd()` y los `var_dump()` cuando terminéis.



💡 Cuando trabajéis en proyectos grandes, existe **Xdebug**, una extensión que permite parar el programa en cualquier línea desde VS Code y mirar todas las variables. Por ahora, con `var_dump()` y `dd()` vais sobrados.



CLASE 13 · PUNTO 6

🚀 Excepciones: try, catch y finally

Hay errores que no son culpa del programador: el usuario divide entre cero, un archivo no existe, la base de datos no responde... No podemos evitarlos, pero sí **controlarlos** para que la web no se rompa. Para eso están las **excepciones** [5].

Pensad en un trapecista con red: `try` es el número arriesgado y `catch`, la red. Si algo sale mal dentro del `try`, PHP salta directamente al `catch` en lugar de estrellarse.

04-excepciones.php — php-paso-a-paso/clase13 — Visual Studio Code

04-excepciones.php

```php
<?php
try {
    echo intdiv(10, 0);                // ¡PHP lanza DivisionByZeroError!
    echo "Esta línea no se ejecuta";
} catch (DivisionByZeroError $error) {
    echo "No se puede dividir entre cero.";
}
```

Lanzar nuestras propias excepciones: throw

Nuestras funciones también pueden avisar de que algo va mal con `throw`:

04-excepciones.php — php-paso-a-paso/clase13 — Visual Studio Code

04-excepciones.php

```php
<?php
function retirarDinero(float $saldo, float $cantidad): float
{
    if ($cantidad > $saldo) {
        throw new Exception("Saldo insuficiente: tienes {$saldo} €.");
    }
    return $saldo - $cantidad;
}

try {
    $saldo = retirarDinero(100, 500);
} catch (Exception $e) {
    echo "Operación rechazada: " . $e->getMessage();
} finally {
    echo "Esto se ejecuta SIEMPRE, haya error o no.";
}
```

Pieza	Para qué sirve
`try { }`	El código que puede fallar
`catch (Tipo $e) { }`	Qué hacer si falla. Podéis poner varios `catch` para distintos tipos
`$e->getMessage()`	El texto del error
`finally { }`	Se ejecuta siempre (opcional). Útil para cerrar o registrar algo
`throw new Exception("...")`	Lanzar vosotros una excepción

📄 Ejemplo: `04-excepciones.php`



💡 En la parte de POO veréis que las excepciones son **objetos** y que podéis crear las vuestras propias, como `SaldoInsuficienteException`. Por ahora, con `Exception` es suficiente.



CLASE 13 · PUNTO 7

👍 Errores en vuestro ordenador y en internet

	💻 En vuestro ordenador (desarrollo)	🌐 En internet (producción)
¿Mostrar los errores en pantalla?	**Sí**, todos	**Nunca**: darían pistas a un atacante
¿Guardarlos en un archivo?	Opcional	**Sí**, para poder revisarlos
Configuración	`display_errors = On`	`display_errors = Off` y `log_errors = On`

En Laragon ya tenéis los errores visibles. Para una web publicada, se configura en el `php.ini`, o al principio del script [6]:

05-errores-en-produccion.php — php-paso-a-paso/clase13 — Visual Studio Code

05-errores-en-produccion.php

```php
<?php
ini_set("display_errors", "0");
ini_set("log_errors", "1");
ini_set("error_log", __DIR__ . "/errores.log");

error_log("Esto se guarda en el archivo, no se muestra");
```

Al visitante, un mensaje amable («Ha ocurrido un problema, inténtalo más tarde»). A vosotros, todos los detalles en el registro.

📄 Ejemplo: `05-errores-en-produccion.php`. Ejecutadlo y abrid el archivo `errores.log` que aparece en la carpeta.



CLASE 13 · PUNTO 8

🚀 Ejercicios

- ☐ **01 · Una página más.** Copiad la carpeta `web/` y añadid una página «Sobre nosotros» con su enlace en el menú, marcado como activo cuando estéis en ella. ¿Cuántos archivos habéis tenido que tocar para el menú? → `ej01-web/`
- ☐ **02 · Calculadora.** Un formulario con dos números y una operación (`+`, `-`, `*`, `/`). La función `calcular()`, en un archivo aparte, lanza una excepción al dividir entre cero o con una operación desconocida, y la página la captura y muestra el error. → `calculadora/`

- ☐ **03 · Configuración.** `leerConfiguracion(string $ruta): array` lee un archivo JSON. Si no existe, lanza `RuntimeException`; si el JSON está mal escrito, deja que `json_decode()` lance `JsonException` (pista: opción `JSON_THROW_ON_ERROR`). Probadla con un archivo bueno, uno roto y uno que no existe. → `ej03-leer-config.php`



🏆 **Reto:** cread un error a propósito en cada uno de los tipos de la tabla del punto 4 y, **antes de ejecutar**, escribid qué mensaje creéis que va a salir. Leer errores es una habilidad que se entrena.



CLASE 13 · PUNTO 9

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
`require_once`	Para funciones y configuración: se carga una sola vez
`require`	Para trozos de HTML: cabecera y pie
`__DIR__`	Rutas completas que no se rompen: `__DIR__ . "/includes/..."`
Plantillas	Cabecera y pie comunes; cada página, solo su contenido
Leer errores	Tipo → qué ha pasado → archivo y línea
Depurar	No adivinéis: `var_dump()` o `dd()` y acotad
Excepciones	`try` / `catch` / `finally`, y `throw` para lanzar las vuestras
Producción	Errores ocultos al visitante y guardados en un archivo de registro

📚 Fuentes y documentación oficial

1. Manual de PHP: require_once
2. Manual de PHP: constantes mágicas
3. Manual de PHP: constantes predefinidas de errores
4. Manual de PHP: var_dump
5. Manual de PHP: excepciones
6. Manual de PHP: configuración de errores en tiempo de ejecución